

InterviewPal — مستند پیاده‌سازی بک‌اند

API اپلیکیشن تمرین مصاحبه فنی برای برنامه‌نویسان فارسی‌زبان

مورد	مقدار
فناوری	ASP.NET Core (کنترلر)، EF Core 10، NET 10 با SQLite (توسعه و تست) و PostgreSQL (تولید)
ریپو	github.com/nasibehash/interviewPal-backend
API زنده	interviewpal-backend.onrender.com (/health ، /api/technologies)
فرانت‌اند	interview-pal-frontend-sable.vercel.app (ریپوی interviewPal-frontend)
وضعیت	فاز ۲: بانک سؤال، درس‌ها، تمرین و ارزیابی، همراه حساب کاربری و ذخیره پیشرفت روی سرور

۱. هدف

بک‌اند دو منبع محتوا را به فرانت‌اند می‌دهد و تمرین را ارزیابی می‌کند:

- **بانک سؤال:** ۳۰۰ سؤال (۵۰ سؤال برای هر یک از شش فناوری: Angular، JavaScript، TypeScript، React، Next.js و NET). در سه سطح و چهار نوع سؤال. هر سؤال فقط یک سؤال نیست؛ جواب کوتاه برای گفتن در مصاحبه، توضیح کامل، اشتباه رایج و سؤال بعدی مصاحبه‌کننده را هم دارد.
- **درس‌ها:** ۲۴ درس (۱۴ الگوریتم و ۱۰ الگوی طراحی) با مثال واقعی، یک پیاده‌سازی برای هر فناوری و چهار تمرین. محتوا فارسی است و اصطلاح‌های فنی و کدها انگلیسی می‌مانند.

۲. پشته فناوری و تصمیم‌های اصلی

حوزه	انتخاب	دلیل
پلتفرم	ASP.NET Core، NET 10 با کنترلر	ساختار ساده و آشنا برای API
پایگاه داده	EF Core 10؛ Database:Provider، SQLite (پیش‌فرض) یا Postgres	SQLite بدون سرویس جدا برای توسعه و تست؛ PostgreSQL برای تولید چون دیسک Render رایگان پایدار نیست
ساخت پایگاه	SQLite: EnsureCreated؛ PostgreSQL: migration (Migrate) هنگام شروع	جدول‌های کاربر باید با نسخه‌بندی تغییر کنند؛ migration در Infrastructure/Persistence/Migrations است
احراز هویت	JWT (HS256) کوتاه‌عمر + refresh token در کوکی	توکن دسترسی ۱۵ دقیقه؛ refresh token چرخشی و فقط hash آن در پایگاه ذخیره می‌شود
رمز عبور	PasswordHasher (PBKDF2) ASP.NET)	بدون پکیج اضافه؛ salt و تعداد تکرار داخل hash است

حوزه	انتخاب	دلیل
الگوی درخواست	سرویس‌های ساده، بدون MediatR	MediatR مجوز تجاری دارد و برای این اندازه لازم نیست
خطاها	RFC 7807 ProblemDetails با IExceptionHandler	قالب استاندارد؛ فرانت‌اند متن detail را نشان می‌دهد
محتوای درس‌ها	فایل در حافظه (LessonCatalog)	محتوای ثابت است؛ تغییر schema پایگاه لازم نیست
تست	WebApplicationFactory و xUnit	تست یکپارچه روی برنامه واقعی با SQLite جدا؛ همان تست‌ها روی PostgreSQL هم اجرا شده‌اند
استقرار	Docker روی Render	image چندمرحله‌ای و کاربر غیر root

۳. معماری و ساختار پروژه

وابستگی‌ها به سمت داخل‌اند: `Api → Application`، `Application → Infrastructure` و `Application → Application`. `Domain` سرویس‌های Application فقط به رابط repository وابسته‌اند و پیاده‌سازی آن‌ها در Infrastructure است.

```
src/
  InterviewPal.Domain      entities and enums, no dependencies
  InterviewPal.Application contracts (DTOs), services, repository interfaces
  InterviewPal.Infrastructure EF Core (SQLite), repositories, content seeder, lesson catalog
  InterviewPal.Api         controllers, error handling, composition root
tests/
  InterviewPal.Api.Tests   integration tests + content guard tests
content/
  *.json                  the question bank, one file per technology
  lessons/<lesson-id>/    one folder per lesson: lesson.json + code files
docs/
  backend.md              this document (source)
  InterviewPal-Backend.pdf the same document as PDF
```

۴. دامنه و داده

نوع	توضیح
Technology	Slug (کلید)، نام، CurrentVersion و SupportedFrom؛ بانک سؤال بازۀ این دو نسخه را پوشش می‌دهد
Question	شناسۀ پایدار (مثل angular-signal-basics-output)، سطح، نوع، متن، کد، جواب کوتاه، توضیح، اشتباه رایج، سؤال بعدی، زمان تقریبی، نسخه معرفی، تگ‌ها و ContentHash
Choice	گزینه‌های سؤال‌های چندگزینه‌ای؛ دقیقاً یکی درست است
QuestionReport	گزارش کاربر درباره سؤال غلط، قدیمی، مبهم یا غلط نگارشی

نوع	توضیح
Level	Junior ، Mid و Senior (برای هر فناوری ۱۶ و ۱۸ و ۱۶ سؤال)
QuestionType	Conceptual و ShortAnswer ، CodeOutput ، MultipleChoice
User	ایمیل (یکتا، با حروف کوچک)، نام نمایشی، hash رمز، شمارنده ورود ناموفق و زمان قفل
RefreshToken	hash توکن، انقضا، زمان ابطال و زنجیره جایگزینی؛ استفاده دوباره از توکن چرخیده شده همه نشست‌های کاربر را باطل می‌کند
PracticeHistoryEntry و QuestionStat	تاریخچه تمرین‌ها و آمار هر سؤال برای کاربر وارد شده
LessonExerciseProgress	آخرین پاسخ هر تمرین درس برای کاربر
Lesson (در حافظه)	درس با Kind (Algorithm یا DesignPattern)، پیاده‌سازی‌ها و تمرین‌ها

آمار دسته‌بندی سؤال‌ها در `/api/technologies` برمی‌گردد و فرانت‌اند از آن برای کارت‌های انتخاب فناوری استفاده می‌کند.

۵. محتوا و بارگذاری

۵.۱ بانک سؤال

فایل `content/<technology>.json` شامل اطلاعات فناوری و فهرست سؤال‌هاست. هنگام شروع برنامه، `ContentSeeder`:

- هر سؤال را با شناسه‌اش upsert می‌کند و hash محتوا را نگه می‌دارد؛ سؤال بدون تغییر دست نمی‌خورد، پس شناسه گزینه‌ها بین دو اجرا ثابت می‌ماند.
- سؤال‌های تغییرکرده را به‌روز و سؤال‌هایی را که از فایل حذف شده‌اند پاک می‌کند.
- ابتدا `ContentValidator` را اجرا می‌کند و در صورت خطا برنامه بالا نمی‌آید.

قواعد `ContentValidator`: شناسه یکتا و با پیشوند اسلاگ فناوری باشد، سطح و نوع معتبر باشد، حداقل یک تگ، زمان بین ۱۵ تا ۶۰۰ ثانیه، سؤال گزینه‌ای بین ۲ تا ۶ گزینه با دقیقاً یک گزینه درست، `CodeOutput` حتماً کد داشته باشد و سؤال تشریحی گزینه نداشته باشد.

۵.۲ درس‌ها

هر درس یک پوشه زیر `content/lessons/` است:

```
content/lessons/binary-search/
  lesson.json      texts (Persian), metadata, exercises, one entry per technology
  javascript.js    typescript.ts  angular.ts  react.tsx  nextjs.ts  dotnet.cs
```

- `LessonCatalog` درس‌ها را هنگام شروع در حافظه می‌خواند؛ پایگاه داده درگیر نیست.
- `LessonValidator` بررسی می‌کند: همه شش فناوری پیاده‌سازی دارند، فایل کد موجود و غیرخالی است، نام پوشه با `id` برابر است، الگوریتم‌ها پیچیدگی زمانی و حافظه دارند و هر درس حداقل دو تمرین با دقیقاً یک گزینه درست دارد.

- ترتیب گزینه‌های تمرین با یک shuffle قطعی (seed از شناسه تمرین) ثابت و پراکنده می‌شود؛ نویسنده لازم نیست جای گزینه درست را متعادل کند. شناسه گزینه همان اندیس بعد از shuffle است.
- زبان کد از پسوند فایل می‌آید (`.js` ، `.ts` ، `.tsx` ، `.cs`).

قاعده محتوا

کد درس‌ها اجرا یا type-check شده‌اند: JavaScript، TypeScript و #C اجرا شده‌اند و فایل‌های Angular، React و Next.js با typing‌های واقعی فریم‌ورک بررسی شده‌اند. گزینه درست نباید با طول متن قابل حدس باشد؛ تست‌ها این را کنترل می‌کنند.

۶. API

متد	مسیر	کار
GET	/health	بررسی زنده بودن
GET	/api/technologies	فناوری‌ها، بازه نسخه و تعداد سؤال هر سطح
GET	/api/questions	جست‌وجو (فیلتر <code>level</code> ، <code>technology</code> ، <code>search</code> ، <code>page</code> ، <code>tag</code> ، <code>pageSize</code>)؛ بدون جواب
GET	/api/questions/{id}	یک سؤال همراه جواب و توضیح
POST	/api/questions/{id}/reports	گزارش مشکل سؤال
POST	/api/practice/sessions	ساخت تمرین
POST	/api/practice/questions/{id}/check	ارزیابی یک پاسخ
POST	/api/practice/evaluate	ارزیابی کل تمرین
GET	/api/lessons	فهرست درس‌ها (فیلتر <code>level</code> ، <code>kind</code> ، <code>category</code> ، <code>technology</code>)
GET	/api/lessons/{id}?technology=	درس همراه پیاده‌سازی فناوری انتخابی و تمرین‌ها (بدون جواب)
POST	/api/lessons/{id}/exercises/{exerciseId}/check	ارزیابی یک تمرین درس (برای کاربر واردشده ثبت می‌شود)
POST	/api/auth/register	ثبت‌نام؛ توکن دسترسی در بدنه و refresh token در کوکی
POST	/api/auth/login	ورود
POST	/api/auth/refresh	توکن جدید و چرخش refresh token
POST	/api/auth/logout	ابطال refresh token و پاک‌کردن کوکی
GET	/api/auth/me	کاربر فعلی (نیازمند توکن)

متد	مسیر	کار
POST	/api/auth/change-password	تغییر رمز؛ همه نشست‌ها باطل می‌شوند
POST	/api/auth/delete-account	حذف حساب و همه داده آن (نیازمند رمز)
GET	/api/me/progress	تاریخچه، آمار سؤال‌ها و پیشرفت درس‌ها
POST	/api/me/progress/import	انتقال یک‌باره داده محلی مرورگر
DELETE	/api/me/progress	پاک‌کردن پیشرفت ذخیره‌شده

همه پاسخ‌ها JSON با camelCase هستند و enum به صورت رشته می‌آیند. خطاها به شکل ProblemDetails برمی‌گردند: ورودی نامعتبر 400، چیز پیدانشده 404 و خطای پیش‌بینی‌نشده 500 (بدون جزئیات داخلی).

۷. منطق تمرین

- ساخت تمرین (`PracticeService.StartAsync`): `technologies` و `levels` خالی یعنی «همه». تعداد (`count`) بین ۵ و ۱۰۰ است. انتخاب متوازن است: سؤال‌ها به صورت چرخشی از هر فناوری برداشته می‌شوند تا یک فناوری بر تمرین مختلط غالب نشود، و بعد ترتیب نهایی بُر می‌خورد. پارامتر اختیاری `seed` برای تست‌های قطعی است. اگر سؤال کافی نباشد، کمتر برمی‌گردد.
- زمان: `estimatedMinutes` مجموع زمان تقریبی سؤال‌هاست و فقط در حالت `Interview` فیلد `timeLimitSeconds` هم پر می‌شود.
- ارزیابی یک پاسخ: سؤال گزینه‌ای به `choiceId` و سؤال تشریحی به `knewIt` (درست/غلط) نیاز دارد؛ نبودن آن‌ها یا گزینه متعلق به سؤال دیگر 400 است. پاسخ همراه جواب کامل برمی‌گردد.
- ارزیابی کل تمرین: درصد و تعداد درست، تفکیک به فناوری و سطح، «تگ‌های ضعیف» (درصد زیر ۶۰)، شناسه سؤال‌های غلط (برای تمرین دوباره) و نتیجه تک‌تک سؤال‌ها. پاسخ تکراری برای یک سؤال 400 و سؤال ناشناخته 404 است.
- جواب‌ها قبل از پاسخ دادن فاش نمی‌شوند: سؤال‌ها و تمرین‌های درس بدون جواب می‌آیند.

۷.۱ حساب کاربری و امنیت

- توکن‌ها: توکن دسترسی JWT است و ۱۵ دقیقه اعتبار دارد (`Auth:AccessTokenMinutes`). `refresh token` یک مقدار تصادفی است که در کوکی `httpOnly`، `SameSite=Strict` و `Path=/api/auth` (با `Secure` پشت HTTPS) برای ۳۰ روز می‌ماند. هر `refresh` توکن را عوض می‌کند و در پایگاه فقط `hash` آن ذخیره می‌شود.
- شناسایی سرقت: اگر توکن قبلاً چرخانده شده دوباره بیاید، همه نشست‌های آن کاربر باطل می‌شود. خطای `refresh` در کنترلر مدیریت می‌شود تا کوکی پاک شود و 401 برگردد.
- ورود: پیام خطا برای ایمیل ناموجود و رمز اشتباه یکی است (جلوگیری از شمارش حساب‌ها). بعد از ۵ رمز اشتباه، حساب ۱۵ دقیقه قفل می‌شود. علاوه بر آن هر IP در دقیقه حداکثر ۳۰ درخواست به مسیرهای `auth` دارد (`Auth:RequestsPerMinute`، پنجره ثابت).
- اعتبارسنجی: ایمیل معتبر و رمز ۸ تا ۱۲۸ کاراکتر.

- **IP واقعی:** `ForwardedHeaders` برای دو پراکسی (Render و Vercel) فعال است تا محدودیت نرخ و کوکی `Secure` درست کار کنند.
- **ثبت پیشرفت:** وقتی درخواست توکن معتبر دارد، `evaluate` تمرین را در تاریخچه می‌نویسد و `check` درس نتیجه تمرین را ذخیره می‌کند؛ بدون توکن رفتار همان قبلی است. `import` فقط ورودی معتبر (سؤال‌های موجود، سقف تعداد) را می‌پذیرد.
- **زمان:** همه زمان‌ها UTC هستند (value converter برای SQLite).

۸. پیکربندی

کلید	پیش‌فرض	کار
<code>ConnectionStrings:Default</code>	<code>Data Source=interviewpal.db</code>	مسیر SQLite (در Docker): <code>/data/interviewpal.db</code>
<code>Content:Path</code>	<code><خروجی>/content</code>	پوشه محتوا
<code>Cors:AllowedOrigins</code>	<code>["http://localhost:4200"]</code>	مبدأهای مجاز (سرور توسعه Angular)
<code>HttpsRedirection:Enabled</code>	<code>true</code>	ریدایرکت HTTPS خارج از Development؛ image Docker آن را خاموش می‌کند
<code>Database:Provider</code>	<code>Sqlite</code>	<code>Postgres</code> یا <code>Sqlite</code>
<code>Auth:JwtKey</code>	فقط در Development مقدار پیش‌فرض دارد	کلید امضا؛ خارج از Development باید حداقل ۳۲ کاراکتر باشد و گرنه برنامه هنگام شروع بالا نمی‌آید
<code>/ Auth:AccessTokenMinutes</code> <code>RefreshTokenDays</code>	<code>30 / 15</code>	عمر توکن‌ها
<code>/ Auth:MaxFailedLogins</code> <code>LockoutMinutes</code>	<code>15 / 5</code>	قفل موقت حساب
<code>Auth:RequestsPerMinute</code>	<code>30</code>	سقف درخواست به مسیرهای auth برای هر IP

در Docker، کلیدها با `__` نوشته می‌شوند (مثل `Cors__AllowedOrigins__0`). رشته اتصال به صورت lazy از `IConfiguration` خوانده می‌شود تا تنظیمات تست و میزبان اعمال شود.

۹. تست

۶۵ تست با xUnit:

- **API:** `WebApplicationFactory` برنامه را با پایگاه SQLite جدا برای هر اجرا بالا می‌آورد. فیلترها، صفحه‌بندی، ارزیابی، گزارش‌ها، خطاهای ورودی و پاسخ‌های `ProblemDetails` بررسی می‌شوند.
- **درس‌ها:** فهرست و فیلتر، جزئیات با فناوری انتخابی و نبودن جواب در پاسخ، ارزیابی تمرین و خطاهای آن.

- **حساب کاربری:** ثبت نام و ورود (ایمیل تکراری، رمز کوتاه، قفل حساب)، چرخش و تشخیص استفاده دوباره refresh token، کوکی و پرچم‌هایش، تغییر رمز، حذف حساب، محدودیت نرخ و رد شدن مسیرهای /api/me بدون توکن.
- **پیشرفت:** ثبت تمرین هنگام ارزیابی، آمار سؤال‌ها، نتیجه تمرین درس، انتقال یک‌باره داده محلی و جداسازی داده کاربرها.
- **نگه‌بان محتوا (روی فایل‌های واقعی):** اعتبار فایل‌ها، یکتا بودن شناسه‌ها، شش فناوری، حداقل ۵۰ سؤال برای هر فناوری و پوشش هر سه سطح، نسخه پوشش داده شده، پیاده‌سازی شش فناوری برای هر درس و حداقل ۱۲ الگوریتم و ۱۰ الگو، و قابل حدس نبودن گزینه درست (طول گزینه).

```
dotnet test
```

مجموعه تست با پایگاه SQLite اجرا می‌شود. با تنظیم TEST_POSTGRES_ADMIN روی یک رشته اتصال، همان تست‌ها روی PostgreSQL هم اجرا می‌شوند (برای بررسی migration).

۱۰. اجرا و استقرار

۱۰.۱ توسعه

```
dotnet run --project src/InterviewPal.Api # http://localhost:5161
```

Docker ۱۰.۲

```
docker build -t interviewpal-api .
docker run -p 8080:8080 -v interviewpal-data:/data interviewpal-api
```

برای اجرا با حساب کاربری در خارج از Development باید Auth_JwtKey تنظیم شود. image به‌طور پیش‌فرض SQLite دارد؛ برای PostgreSQL از Database__Provider=Postgres و ConnectionStrings__Default استفاده کنید (رشته key=value یا URL مثل postgres://user:pass@host/db ؛ URL با SSL اجباری تبدیل می‌شود).

image دو مرحله دارد (SDK برای build و runtime ASP.NET برای اجرا)، با کاربر غیر root روی پورت 8080 اجرا می‌شود و پایگاه SQLite در volume با مسیر /data می‌ماند. لایه restore جدا کش می‌شود تا تا وقتی فایل پروژه عوض نشده، دوباره اجرا نشود.

Render ۱۰.۳

- Web Service از نوع Docker، شاخه master، مسیر Dockerfile برابر ./Dockerfile و Health Check روی /health.

- متغیر PORT=8080 (پیش‌فرض Render ۱۰۰۰۰ است و image روی ۸۰۸۰ گوش می‌دهد).
- **حساب‌ها به پایگاه پایدار نیاز دارند.** دیسک پلن رایگان Render موقت است و SQLite با هر deploy پاک می‌شود، پس کاربرها هم پاک می‌شدند. یک PostgreSQL رایگان بسازید (مثلاً Neon یا Supabase، یا Postgres خود Render) و متغیرها را بگذارید: Database__Provider=Postgres، ConnectionStrings__Default (همان URL پایگاه) و Auth_JwtKey (یک رشته تصادفی حداقل ۳۲ کاراکتری؛ با تغییر آن همه نشست‌ها باطل می‌شوند).

- migration هنگام شروع خودکار اجرا می‌شود. سؤال‌ها و درس‌ها هر بار از فایل‌های JSON بارگذاری می‌شوند.
- پلن رایگان بعد از ۱۵ دقیقه بی‌کاری می‌خواهد و اولین درخواست بعدی حدود نیم دقیقه طول می‌کشد.
- فرانت‌اند روی Vercel است و `/api` را با `rewrite` به این سرویس می‌فرستد؛ بنابراین CORS لازم نیست.

۱۰.۴ مستندات

این سند در `docs/backend.md` نوشته می‌شود و `docs/InterviewPal-Backend.pdf` از روی آن ساخته می‌شود. با هر تغییر مهم، README، همین سند و PDF با هم به‌روز می‌شوند.

۱۱. محدودیت‌ها و گام‌های بعدی

موضوع	وضعیت فعلی	گام بعدی
تأیید ایمیل و فراموشی رمز	ندارد	ارسال ایمیل (لینک تأیید و بازنشانی رمز)
محدودیت نرخ	پنجره ثابت در حافظه هر نمونه	ذخیره مشترک (مثل Redis) در صورت چند نمونه
حالت مصاحبه زمان‌دار	زمان فقط در کلاینت اعمال می‌شود	محاسبه زمان در سرور
مرور فاصله‌دار	ندارد	زمان‌بندی مرور بر اساس عملکرد قبلی
ویرایش محتوا	فایل JSON	پنل ادمین برای سؤال‌ها، درس‌ها و بررسی گزارش‌ها
مصاحبه‌کننده هوشمند	ندارد	پاسخ متنی/صوتی با بازخورد